

Progetto ingegneria del software

SnakeAI

Davide Fiorentini, 1091979

Manuela Fiorentini, 1092216

Federico Magli, 1093980

Brian Gervasoni, 1093188

Obiettivo

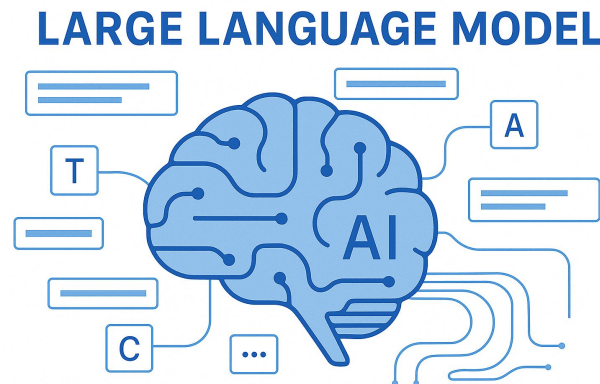
Aiutare l'utente finale a comprendere meglio il funzionamento di un'Intelligenza Artificiale di base sviluppandone e allenandone una che possa giocare al classico gioco snake

Difficoltà incontrate

- Imparare ad usare diagrams.net.
- Imparare ad usare JUnit Tests.
- Imparare ad usare branch, issue, pull requests, eventuali merge.
- Convertire i vari modelli in un codice funzionante, mantenendo il comportamento modellato.

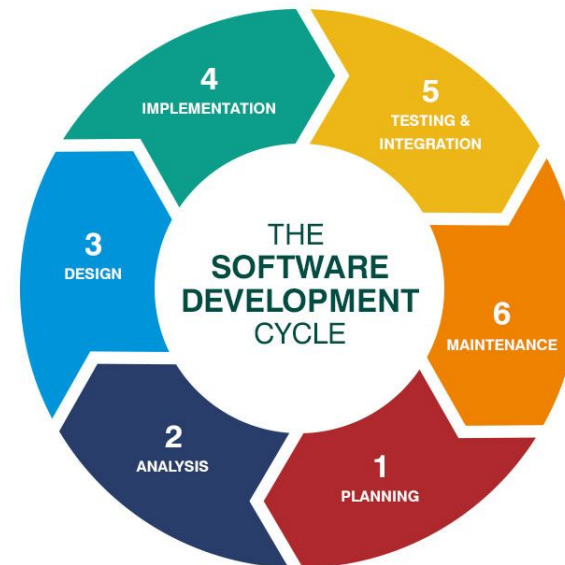
Paradigma di programmazione/modellazione utilizzato e tools

- Paradigma di programmazione ad oggetti, modellando con UML i diagrammi di classe, di flusso e di sequenza.
- Linguaggio usato: Java(100%).
- Eclipse, Google Drive, GitHub, LLM, Draw.io (per il Diagramma delle Classi abbiamo usato Papyrus)



Software life cycle

- Abbiamo usato un SDLC di tipo agile, prendendo spunto dal XP e Scrum.
- In particolare abbiamo dato tanta importanza all'adattamento, durante l'implementazione, della vecchia documentazione ai problemi che sorgevano, mantenendo sempre un overlay dato da esso minimo.



Software configuration management

- git + piattaforma GitHub
- l'uso di GitHub come version control system



git

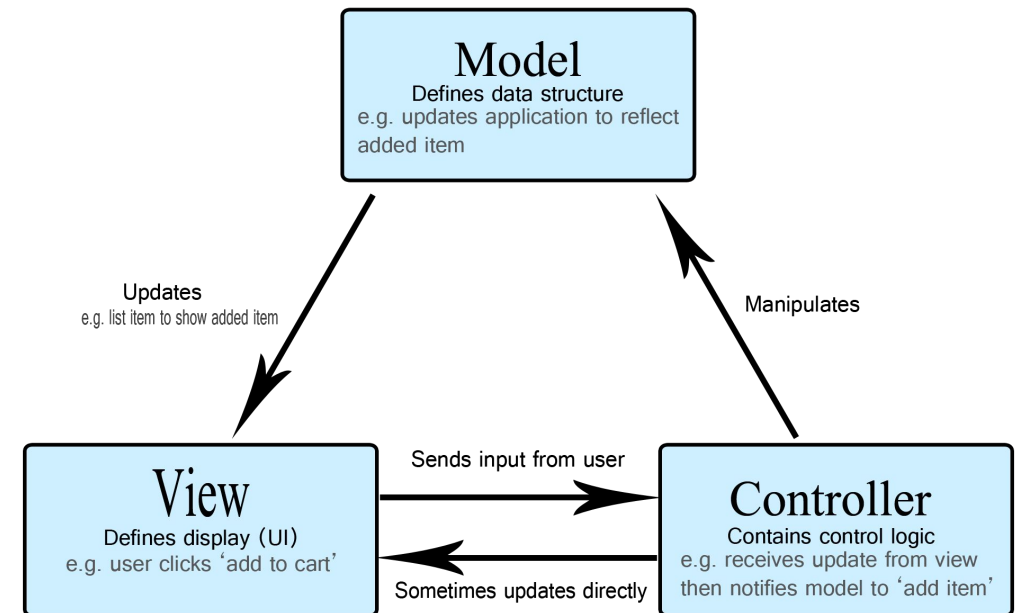


Requisiti

- I requisiti del modulo AI sono stati estratti dal paper di openAI sul PPO. Quelli di snake sono stati condizionati dal regolamento standard di snake e l'interfaccia dal tipo di architettura necessaria per rappresentare il flusso di dati (MVC).
- Specificati nella specifica dei requisiti

Architettura

- MVC per la comunicazione tra moduli.
- Architettura a stack con 3 livelli:
 - 1- interfaccia.
 - 2- modulo AI.
 - 3- file manager.



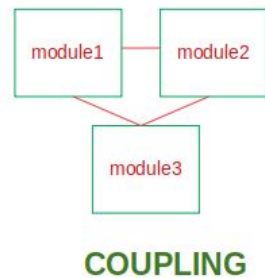
Design pattern e qualità:

- Design pattern utilizzati:

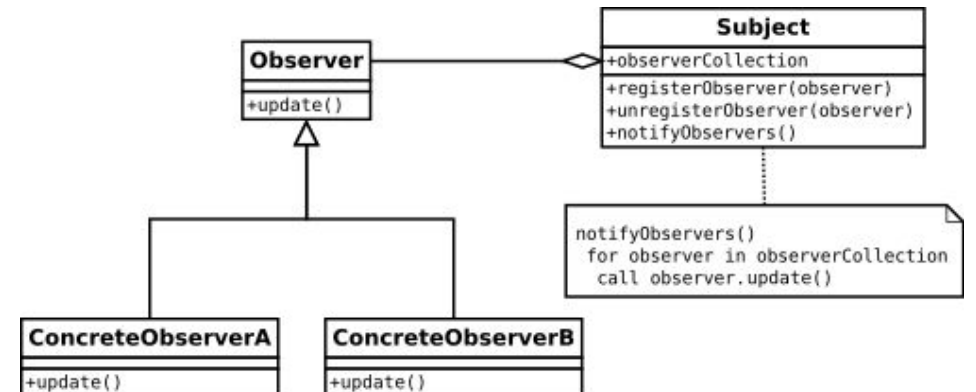
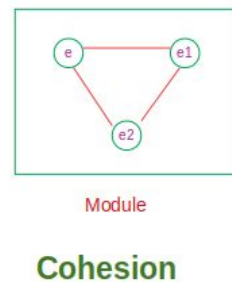
MVC + Observer,
delegation.

Metriche di qualità del progetto:

Coesione e accoppiamento



Vs



Thread + UI + Game + Boxes

Analysis of snakeAI

General Information

Total lines of code: 1669

Number of classes: 25

Number of packages: 4

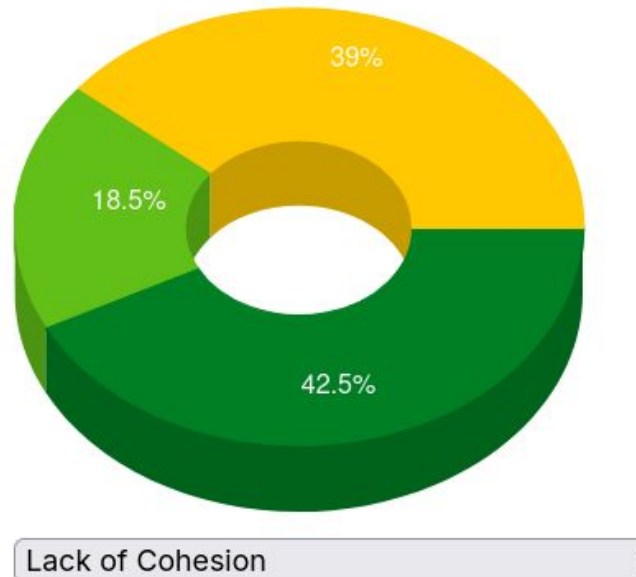
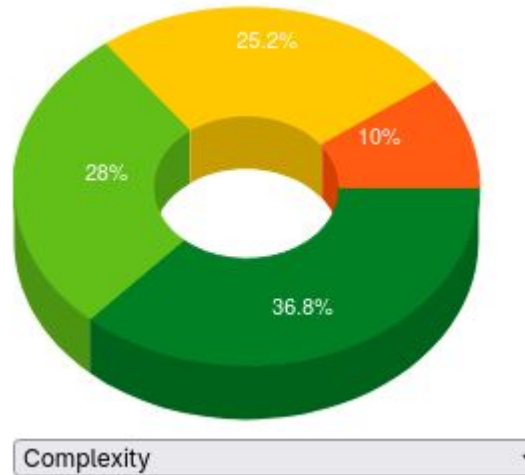
Number of external packages: 32

Number of external classes: 108

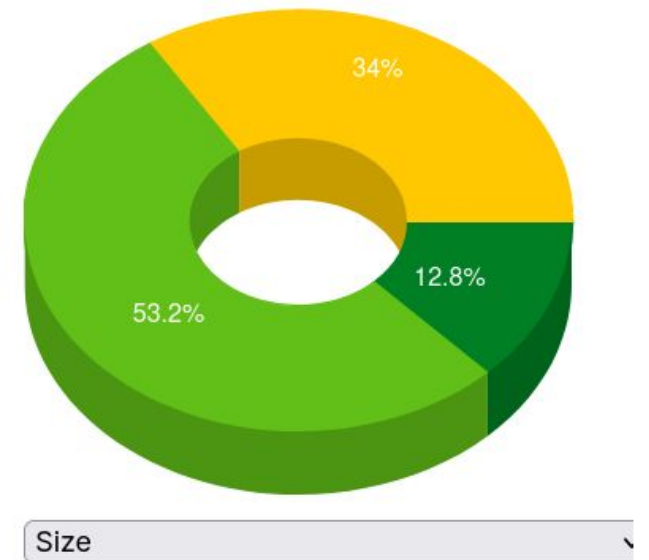
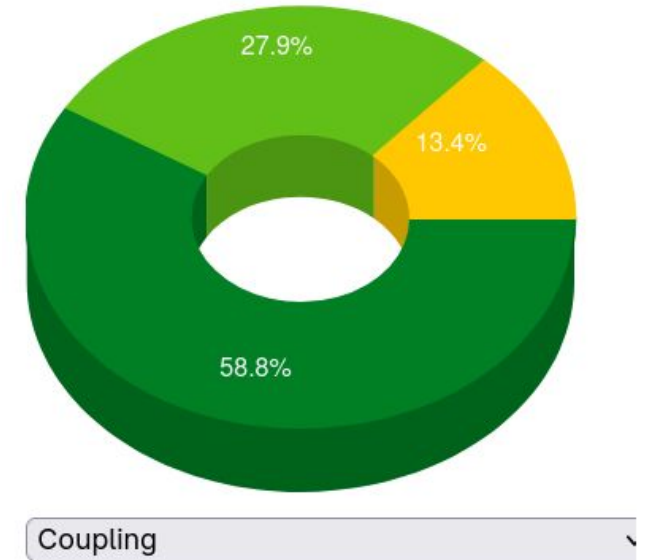
Number of problematic classes: 1

Number of highly problematic classes: 0

- Very High
- High
- Medium-high
- Low-medium
- Low



CodeMR (eclipse plugin)



Blocco AI (AI + hyperparameters + model + thread + tools)

Analysis of snakeAI

General Information

Total lines of code: 1301

Number of classes: 22

Number of packages: 5

Number of external packages: 29

Number of external classes: 97

Number of problematic classes: 2

Number of highly problematic classes: 0

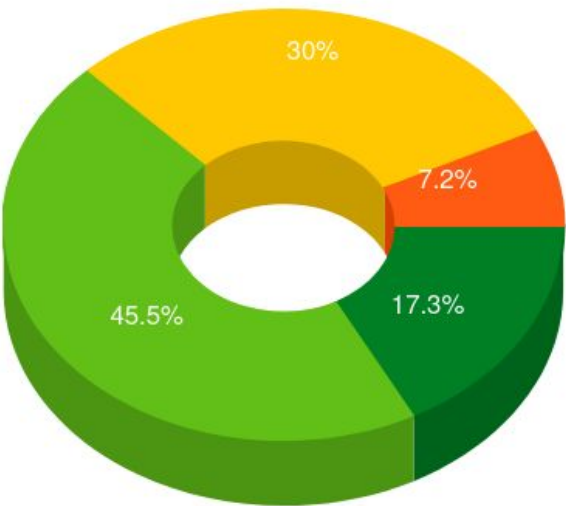
● Very High

● High

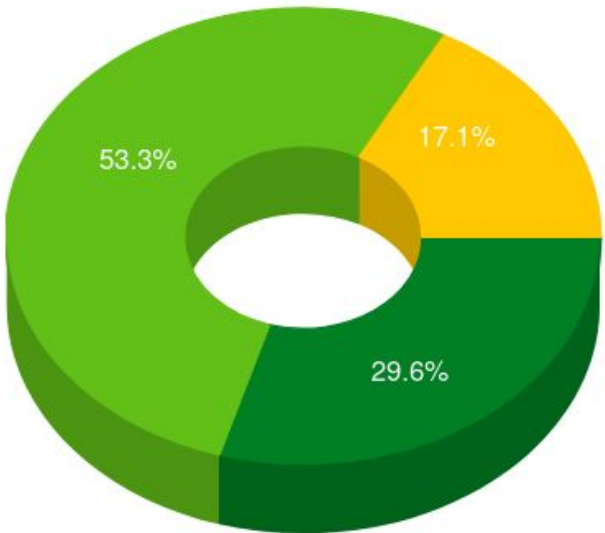
● Medium-high

● Low-medium

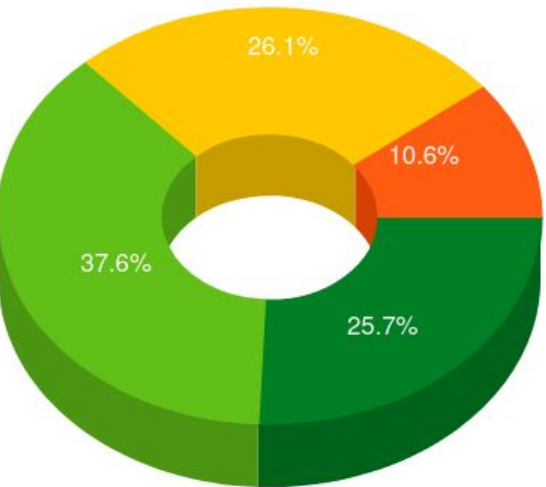
● Low



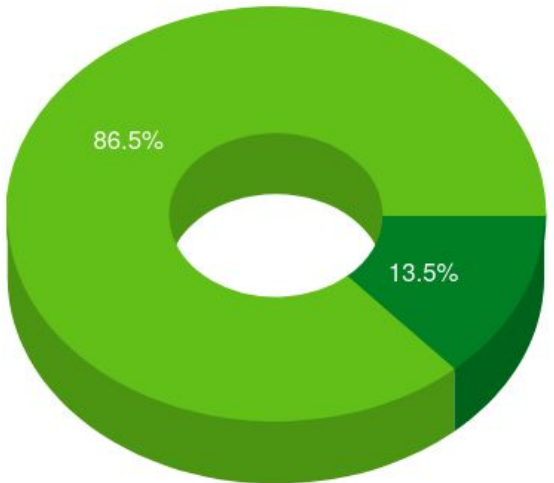
Complexity



Coupling



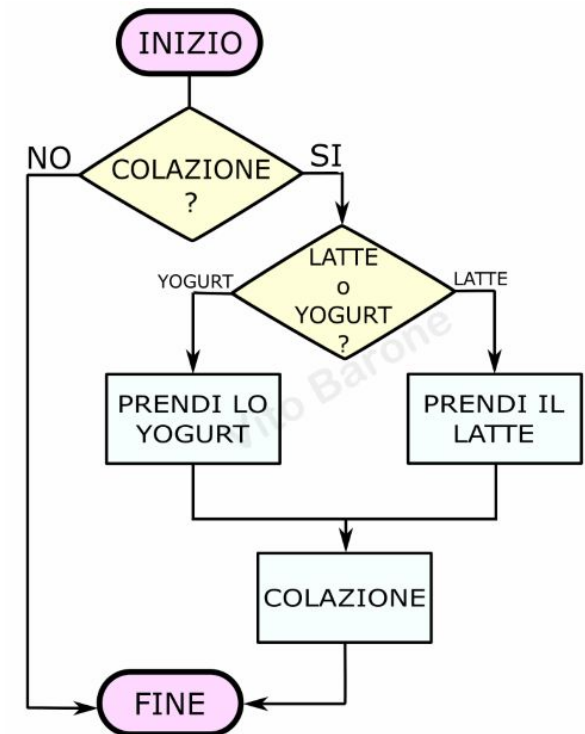
Lack of Cohesion



Size

Modellazione

- Diagramma delle Classi
- Diagramma di Flusso
- Diagramma di Sequenza
- Diagramma dei Casi d'uso
- Diagramma delle Attività
- Diagramma di Stato
- Diagramma del Tempo
- Diagramma di Comunicazione
- Diagramma dei Componenti
- Diagramma dei Pacchetti



Implementazione

Appoggiati ad implementazione manuale

Non implementato: ottimizzazione

Testing

- testing
 - Lato logica di gioco: prima testing manuali, dopo JUnit Tests
 - Lato AI: JUnit Test per le componenti critiche, test manuale per i test tra componenti.
- Fondamentali per la rilevazione di problemi logici.
- Il codice, data la sua astrazione, è abbastanza resistente alle modifiche.



